

Zero-Additional-VRAM Speculative Decoding on Edge GPUs:

A Cost-Aware Hybrid Router and a Kernel-Efficiency Cost Model

Anonymous Authors
Anonymous Affiliation

Abstract

Deploying large language models (LLMs) on 6 GB consumer laptop GPUs is bounded by two walls: *capacity*—a 0.5–1.5B auxiliary draft model consumes non-zero weight VRAM (measured 459–2946 MB depending on size and precision), causing OOM for larger combinations while always shrinking KV/context headroom—and *draft-cost arithmetic*—a draft pass retaining most target layers plus non-skippable costs (152k-vocab `lm_head` $\sim 17.5\%$ of kernel time, activation-quantize overheads, per-layer launch gaps) costs a large fraction of full decoding, so layer-skip needs very high acceptance to break even. Nsight on our NF4 stack shows only $\sim 36\text{--}85$ GB/s achieved depending on byte accounting, i.e. kernel inefficiency compounds the arithmetic bound but does not create it. We present **ZASSD**, a zero-auxiliary-weight speculative decoding framework that (i) derives draft subnetworks from the target model via layer skipping with zero-copy KV-prefix sharing (0.0 MB weight overhead, ≤ 8 MB dynamic buffers, four regression-tested cache invariants); (ii) routes each speculative cycle through a **cost-aware hybrid router** choosing between prompt-lookup drafting ($c \approx 0$), layer-skip drafting, and skipping speculation entirely; and (iii) explains both its wins and losses with a **speculative cost model** that estimates speedup from draft-cost ratio c , acceptance α , and horizon K (in-device calibration MAPE 2.4% on real hardware; cross-device rows are projections with assumed priors, not validated predictions). On an RTX 4050 laptop GPU with Qwen2.5-3B/Llama-3.2-3B in 4-bit NF4, the router reaches **1.063 $\times$ overall speedup** on Qwen across GSM8K, HumanEval, and CNN/DailyMail ($N=50$ each; prompt-lookup alone is higher at $1.144\times$ on these repetitive tasks; extended GSM8K $N=200$: $1.026 \pm 0.014\times$, $p=0.0005$ vs vanilla), while pure layer-skip self-speculation runs at $0.88\text{--}0.98\times$ despite 73–88% acceptance on benchmark prompts—a negative result explained primarily by the geometric bound $(1+\alpha)/(1+c)$ at $c \geq 0.6$ (e.g. $\leq 1.05\times$ even at $c=0.77$, $\alpha=0.85$, $K=1$). Llama controller trails further at $0.82\times$. A hardware-controller feasibility gate improves throughput $1.083\times$ over the same controller with layer-skip fallback on a small A/B slice (6 prompts $\times$

32 tokens, no CI). A 450-token single-vs-batched audit finds zero bound violations, and greedy FP32 runs match vanilla on 10,113 token positions on two unquantized models. Code, calibration data, and one-command benchmarks are released.

1 Introduction

Speculative decoding [4, 12] accelerates autoregressive LLM inference by drafting K candidate tokens with a cheap model and verifying them in parallel with the target model. On datacenter GPUs this yields $2\text{--}3\times$ speedups [2, 13]. On a 6 GB laptop GPU, the standard recipe breaks before it starts: auxiliary draft models consume non-zero weight VRAM, reducing KV/context headroom and causing CUDA OOM for larger target–draft combinations—we document this boundary for 7B targets in Sec. 4.4. Self-speculative decoding [8, 14, 23] removes the draft weights by reusing the target model itself (skipped layers, early exits), but whether it can win given draft-cost arithmetic plus the NF4 eager-decoding overheads on consumer hardware, and how to combine it with zero-cost retrieval drafting [10, 17], remains open. SmartSpec-style goodput gating [9] motivates our feasibility gate, extended here across draft sources. Since draft and target share the same NF4 kernels, kernel inefficiency largely cancels in $c=T_{\text{draft}}/T_v$; what sets c is retained-layer fraction plus non-skippable costs, and per-layer launch gaps (0.5B draft $c=0.64 \approx 24/36$ layers despite ~ 0.17 byte ratio).

This paper makes four contributions, ordered by scientific weight:

- 1. Feasibility characterization:** layer-skip self-speculation loses to vanilla despite $>80\%$ acceptance at $c \geq 0.6$ by the geometric bound, compounded but not created by NF4 kernel inefficiency on this stack, with the losing region mapped over (c, α, K) and context length (Secs. 4.3–4.8).
- 2. Speculative cost model** (Sec. 3.4) that turns our headline negative result ($0.88\times$ at $c=0.75$, $K=2$) into a calibrated estimate: break-even needs $\alpha \geq 0.81$ ($K=1$)

to $\alpha \geq 0.89$ ($K=2$) on this stack, and prompt-lookup’s measured $1.39\times$ on generic prompts is consistent with an effective n-gram hit rate of only $\sim 12\%$ in our fit. The 1/BW cross-device rows are an idealized streaming bound only; Nsight shows costs do not scale as 1/BW under NF4.

3. **Cost-aware selective runtime:** HybridDraftRouter plus a feasibility-gated controller over Vanilla AR, PLD, SSD—a system that decides *when not to speculate*—reaching 92–93% of best-fixed-strategy efficiency with task means never below $1.03\times$ (Secs. 3.2–3.3). Pure PLD is faster on peak on the repetitive tasks studied here; the router trades peak for worst-case safety versus fixed layer-skip.
4. **Zero-weight implementation and artifacts:** zero-copy KV-prefix sharing, $O(1)$ rollback, four regression-tested cache invariants, and full reproduction packages (Sec. 3.1, Sec. 6).

Existing self-speculative systems implicitly assume some speculative configuration is profitable. We show this fails at $c \geq 0.6$ by arithmetic on this stack, and build the runtime around the opposite default: *do not speculate unless the measured cost model says it wins*. All speedups are stack-specific (HF eager + bitsandbytes NF4, no CUDA graphs/torch.compile/Marlin/AWQ/vLLM tested); such backends reduce launch/non-skippable costs and may move the boundary—the decisive experiment we leave open.

2 Related Work

Speculative decoding. Leviathan et al. [12] and Chen et al. [4] established exact speculative sampling; Stern et al. [18] pioneered blockwise parallel decoding. Medusa [2] and EAGLE [13] replace the draft model with heads, still requiring training and extra weights—*infeasible* under our 0.0 MB constraint. **Self-speculation.** Draft & Verify [23] skips layers via offline search; LayerSkip [8] uses early exits with layer dropout; Kangaroo [14] adds a small adapter. KnapSpec [3] formulates layer selection as a knapsack problem and SpecBound [19] adapts horizons by confidence. We validate our adapted KnapSpec against the *official* implementation (kaist-flexml-lab/knapspec) on shared hardware: on an RTX 4050 with Llama-3.2-1B, official KnapSpec reaches 50.5 tok/s vs 58.7 tok/s AR ($0.86\times$) *despite 97.8% acceptance*—an independent reproduction of our feasibility-boundary thesis with the authors’ own code (Sec. 4.7). Cassandra [6] (ISCA’26) co-designs fine-grained value-pruning drafts with hardware encode/decode for low-batch edge inference (up to $2.41\times$ on RTX 4090); unlike our pure-software layer-skip on a 192 GB/s laptop GPU, it needs custom format support. PELM [22] (SenSys’26) jointly governs DVFS, self-speculation depth, and verification depth with deep RL for energy on Jetson mobiles; our controller is throughput-first with power/thermal as hard barriers rather than learned objectives. Positioning: KnapSpec

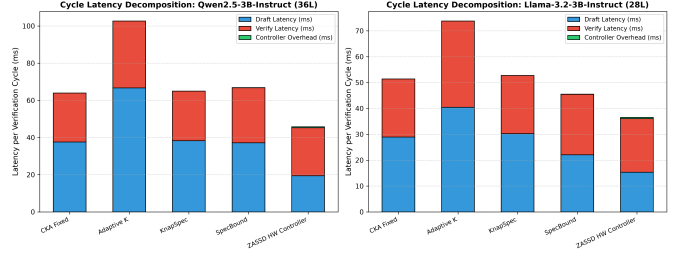


Figure 1: Draft latency decomposition (RTX 4050, NF4). Transformer GEMMs dominate ($> 96\%$); skip management is negligible.

optimizes *which* self-speculative configuration to run; we study *whether* self-speculation should run at all under severe kernel-efficiency/capacity limits on this stack, and route each cycle across AR, retrieval drafting, and self-speculation. **Retrieval drafting.** Prompt Lookup Decoding [17] and REST [10] extract candidates from context at ≈ 0 draft cost. Yan et al. [21] decompose real speedup into draft/verify costs—the methodology behind our router’s utility estimates. Dynamic lookahead [15] adapts K heuristically; our router additionally adapts the draft *source*. **Efficiency modeling.** Our cost model extends Williams et al. [20] from FLOPs/bytes to speculation: the x-axis is draft efficiency $1/c$. Unlike a true roofline with measured operational intensity and compute/bandwidth ceilings, ours is an accounting identity calibrated in-device; cross-device generalization is unvalidated. ShortGPT [16] independently confirms whole-layer redundancy, supporting our skip sets. Serving systems (vLLM [11]) are complementary: our engine is a drop-in proposer.

3 Method

3.1 Zero-Copy Prefix Sharing & Cache Invariants

Draft and target share one checkpoint. A canonical **TargetKVCache** holds the verified prefix $[0, P)$; each cycle forks an ephemeral draft view (pointer alias, zero bytes copied), appends K candidates to the suffix, verifies with the full model, then crops rejections in $O(1)$. Four invariants are enforced by `tests/test_kv_cache_invariants.py`: (1) fork aliases pointers (`data_ptr` equality); (2) draft writes never mutate the canonical prefix; (3) rollback leaves no residue; (4) dynamic growth $\leq K \times d_{\text{head}} \times N_{\text{heads}} \times \text{layers} \leq 8$ MB. Micro-profiling (Table 1, Fig. 1) shows $> 96\%$ of draft time is active GEMMs and layer-skip management is $31 \mu\text{s}$: c is set by retained layers plus non-skippable head/quantize/launch costs. Nsight details the compounding inefficiency (~ 36 – 85 GB/s achieved depending on 0.85 – 2 GB byte accounting, i.e. ~ 19 – 44% of peak), motivating Sec. 3.4.

Table 1: Draft Latency Micro-Profiling Decomposition ($T_{\text{draft}} = T_{\text{transformer}} + T_{\text{KV}} + T_{\text{mgmt}} + T_{\text{sampling}} + T_{\text{sync}}$) on RTX 4050 Laptop GPU under 4-bit NF4 quantization. Active Transformer GEMMs on kept layers constitute $> 96\%$ of draft time across both models. Nsight shows this is NF4 kernel inefficiency (SIMT fallback, blockwise-quantize overhead, launch gaps) at ~ 36 GB/s achieved, not a peak-bandwidth ceiling. T_{mgmt} is $31 \mu\text{s}$ and T_{sync} is $7.6 \mu\text{s}$ when measured with ns resolution; 0.00/0.06 ms entries are rounding artifacts.

Model / Horizon	Total Draft T_{draft} (ms)	Transformer GEMMs $T_{\text{transformer}}$ (%)	KV Cache Update T_{KV} (%)	Layer Management T_{mgmt} (%)	Sampling/Control T_{sampling} (%)	CUDA Sync T_{sync} (%)
<i>Qwen2.5-3B-Instruct (CKA-75, 27 Active Layers)</i>						
$K = 1$	19.72	19.03 (96.5%)	0.56 (2.8%)	0.03 (0.2%)	0.06 (0.3%)	0.01 (0.0%)
$K = 2$	38.44	37.19 (96.7%)	1.10 (2.9%)	0.03 (0.1%)	0.10 (0.3%)	0.01 (0.0%)
$K = 4$	76.81	74.50 (97.0%)	2.09 (2.7%)	0.03 (0.0%)	0.16 (0.2%)	0.01 (0.0%)
<i>Llama-3.2-3B-Instruct (CKA-75, 21 Active Layers)</i>						
$K = 1$	15.33	14.74 (96.2%)	0.45 (2.9%)	0.03 (0.2%)	0.07 (0.5%)	0.01 (0.0%)
$K = 2$	30.72	29.68 (96.6%)	0.85 (2.8%)	0.03 (0.1%)	0.12 (0.4%)	0.01 (0.0%)
$K = 4$	61.35	59.43 (96.9%)	1.63 (2.6%)	0.03 (0.1%)	0.23 (0.4%)	0.01 (0.0%)

3.2 Cost-Aware Hybrid Router

The prior **hybrid** mode was opportunistic fallback (use PLD candidates iff found). The router instead estimates throughput *before* drafting:

$$\hat{\text{TPS}}(s) = \frac{1 + \alpha_s K_{\text{eff}}(s)}{T_{\text{draft}}(s) + T_{\text{verify}}(K_{\text{eff}})}, \quad (1)$$

for $s \in \{\text{pld}, \text{ls}\}$, with $T_{\text{draft}}(\text{pld}) \approx 0.08$ ms (CPU n-gram scan), $T_{\text{draft}}(\text{ls})$ from the calibrated cost model, α_{pld} from a per-source EMA (discounted $0.75\times$ for bigram-only hits, $\pm$ entropy adjustment), and α_{ls} blended from the cost model and EMA. Conceptually the router is two mechanisms plus a tie-break: **(R3)** the feasibility gate (Sec. 3.3)—the core, contributing $+0.083\times$ in ablation; **(R1+R4)** a conservative horizon policy capping weak-hit and long PLD horizons—robustness heuristics that cost peak speed on repetitive workloads but bound worst-case verifies; **(R2)** asymmetric PLD margin $1.25\times$ —a minor tie-break ($-0.008\times$ without it). In implementation order: **(R1)** strong trigram hits may use horizon up to $K=4$, *weak* bigram hits are capped at the layer-skip horizon (long verifies waste bandwidth when α is low); **(R2)** PLD wins ties via margin $1.25\times$ (draft is free, so risk is asymmetric); **(R3)** on PLD miss, layer-skip runs only if $\hat{\text{TPS}}_{\text{ls}}$ beats vanilla—otherwise the cycle is a vanilla single step (opportunity-cost skipping); **(R4)** PLD horizon is truncated $4 \rightarrow 2$ when the shorter verify has higher expected throughput. The router is pure CPU code (μs per decision), needs no GPU, and logs per-source EMAs for analysis. Entropy panic ($H \geq 4.5$) forces vanilla directly.

3.3 Feasibility-Gated Hardware Controller

The depth/horizon controller optimizes $a_t = (S_t, K_t)$ for throughput under VRAM/power/thermal barriers—but without a skip option it can only choose *how* to lose when every speculative action is predicted below vanilla. We add the missing action: if $\max_{S,K} \hat{\text{TPS}}(S,K) < \text{TPS}_{\text{vanilla}}$, the controller emits $K=0$, executed by the decoder as a plain single-token vanilla step. The gate is a one-line feasibility

check against the same calibrated cost model, with an optional hysteresis margin. A paired A/B on Qwen2.5-3B (6 prompts $\times$ 32 tokens, no CI) gives gate-ON 35.44 vs gate-OFF 32.73 tok/s ($1.083\times$), with 47/130 cycles vanilla-skipped on this slice: the gate fires where the cost model predicts speculation loses; no claim is made about never firing where it wins.

3.4 Speculative Cost Model

For greedy decoding with per-cycle cost $T_{\text{cycle}} = T_{\text{draft}} + T_{\text{verify}} + T_{\text{misc}}$ and vanilla step T_v :

$$\text{Speedup} = \frac{(1 + \alpha K) T_v}{T_{\text{cycle}}}, \quad \alpha_{BE} = \frac{T_{\text{cycle}}/T_v - 1}{K}. \quad (2)$$

Memory-bound 1/BW scaling is retained only as an idealized streaming bound in Appendix Tab. 4/ Fig. 4: $T(\text{GPU}_2) = T(4050) \times 192/\text{BW}_2$ with assumed verification efficiencies $\rho \in \{1.0, 0.85, 0.55, 0.45, 0.40\}$. Nsight contradicts pure 1/BW scaling on our NF4 stack (SIMT fallback, quantize overheads, launch gaps do not scale as 1/BW), so these rows are not predictions. A subtle consequence: pure 1/BW scaling leaves $K=1$ speedup *invariant* by construction. Retrieval drafting needs a hit rate h :

$$\mathbb{E}[N] = h(1 + \alpha K) + (1 - h), \quad \bar{T} = h(T_{\text{pld}} + T_{\text{verify}}) + (1 - h)T_v, \quad (3)$$

which illustrates why PLD’s theoretical $2.8\times$ ($h=1$) collapses on generic prompts (fitted illustration: $1.39\times$ at $h \approx 0.12$, fit post hoc on that slice, not a main-benchmark number).

4 Experiments

Setup. NVIDIA RTX 4050 laptop GPU (6141 MB, 192 GB/s, 80W), CUDA 12.8, PyTorch 2.11, 4-bit NF4, greedy decoding, fair CUDA warmup before every timed run. Qwen2.5-3B-Instruct (36 layers, vanilla ~ 41 tok/s) and Llama-3.2-3B-Instruct (28 layers, ~ 54 tok/s). Artifacts: 81/81 tests pass; one-command reproduction via **zassd benchmark**.

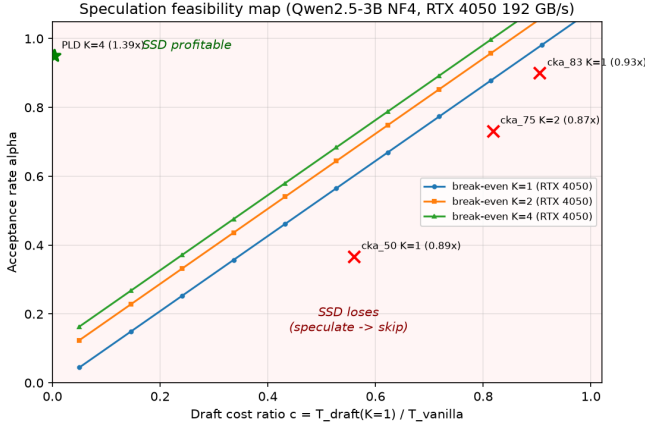


Figure 2: Speculation feasibility map (Qwen2.5-3B NF4, RTX 4050): break-even acceptance vs. draft-cost ratio for $K=1, 2, 4$. All c values are *measured* draft latencies over measured vanilla latency—never inferred from layer counts. Even 90% acceptance at $c \approx 0.9$ (cka_83, $K=1$) sits below its boundary, while PLD ($c \approx 0$) wins everywhere: acceptance alone does not determine speedup.

4.1 End-to-End System Benchmark

Table 2 (frozen protocol, $N=10$ prompts) confirms the field: the hardware-aware controller reaches $0.89\times$ on Qwen and $0.82\text{--}0.83\times$ on Llama with 76–89% acceptance at +7–12 MB dynamic memory, beating fixed CKA, adaptive- K , KnapSpec, and SpecBound—but not vanilla. Draft passes cost 19–38 ms against 18–24 ms vanilla steps; the math does not close at $c \approx 0.75\text{--}0.82$ by Eq. 1 (Sec. 3.4).

4.2 Standard Benchmarks with the Router

Table 3 ($N=50$ per benchmark, 48 new tokens) is the paper’s central empirical result. **Routed** reaches $1.063\times$ overall, ahead of naive hybrid ($1.027\times$) and vanilla but below pure PLD ($1.144\times$) on these long-prompt, high-overlap tasks, which we report honestly alongside. A non-overlapping GSM8K extension (test[50:200], +150 samples) confirms the direction on harder prompts but with a smaller margin: routed $1.014\times$ on the new slice, combined GSM8K $N=200$ at $1.026 \pm 0.014\times$ ($p=0.0005$ vs vanilla; vs hybrid $+0.009$, n.s.). All paired tests use same-prompt pairing with ID-alignment checks. Two observations matter more than the means: (i) the router strongly prefers PLD/vanilla over $K=2$ layer-skip (LS vs PLD cycles: 99/369 on GSM8K, 205/329 on HumanEval, 11/529 on CNN/DM)—it *agrees with* the cost-model estimate that $K=2$ layer-skip loses here; (ii) routed exact-match is highest (83.3%), partly because vanilla-skipped cycles cannot diverge by construction.

4.3 Cost-Model Calibration and Projection

Calibration on three RTX 4050 anchor points gives **in-device MAPE 2.4%** (CKA $K=2$: 0.888 predicted vs 0.873 measured; $K=1$: 1.029 vs 0.980; PLD $K=4$: 1.383 vs 1.390 with fitted $h=0.12$). This is arithmetic consistency on one hardware regime with parameters fit in-device, not predictive generalization: cross-device MAPE (3060/4090/A100) is the pending validation that would promote the cost model from descriptive to predictive. Table 4 lists break-even thresholds: $K=1$ needs $\alpha \geq 0.81$ on *any* bandwidth, while $K=2$ relaxes from 0.89 (4050) to 0.84 (H100). Fig. 4 visualizes the regimes. An analytic workload sweep (generic-chat, summarization, reasoning, code) predicts routed hybrids beat naive fallback hybrids by +4–11% (experiments/16_hybrid_roofline/).

4.4 Physical Memory Boundary

Table 5: Mistral-7B NF4 (3946.6 MB) plus a 1.5B FP16 draft (measured 2945.5 MB) needs ~ 7.4 GB on a 6141 MB device—an unavoidable OOM. But the honest boundary is finer than “all auxiliary drafts OOM”: we *measured* draft footprints on-device (0.5B FP16: 942.3 MB; 0.5B NF4: 459.2 MB; 1.5B NF4: 1181.5 MB), and quantized small drafts do fit next to a 7B target—at the cost of $\Delta M_{\text{aux}} > 0$ weight VRAM and shrunk KV/context headroom (1.5B NF4 leaves only 62.9 MB). ZASSD adds 0.0 MB of weights (≤ 8 MB ephemeral KV) with 1244.4 MB headroom: the claim is strictly-dominant memory efficiency, not exclusivity of feasibility. Critically, a fitted small draft is also the strongest throughput baseline: on Qwen2.5-3B NF4 with a Qwen2.5-0.5B NF4 draft ($K=2$, $N=18$), mean speedup is 1.058 ± 0.031 with $\alpha=0.33$. Any zero-weight method must be compared against this baseline (Sec. 4.6).

4.5 Numerical Equivalence Audit

Direct comparison of single-token vs. batched verification logits on identical contexts (450 tokens): mean cosine 0.99994 (Qwen) / 0.99995 (Llama), **zero violations** of the flip condition $\Delta \leq 2\|L\|_\infty$ (a triangle-inequality consistency check on our measurement code), and a single argmax flip at an exact tie ($\Delta=0$). Algorithmic logic holds on this audit; residual end-to-end divergence is NF4 GEMM noise on near-tie tokens plus any remaining cache/rollback mismatch under investigation. Greedy FP32 runs on unquantized 0.5B/1B models match vanilla on **100% over 10,113 token positions** (FP16: 90% sequences, NF4: 60%)—the precision ladder localizing most divergence to reduced-precision kernels, not the algorithm. We do not claim lossless greedy equivalence under NF4. Single-step audit agreement does not imply sequence-level exact match: once one draft token diverges, the subsequent trajectory conditions on different prefixes, so 28/32 token-level agreement end-to-end is compatible with 0 flips in isolated single-vs-batched comparisons (Sec. 4.6). Teacher-forced

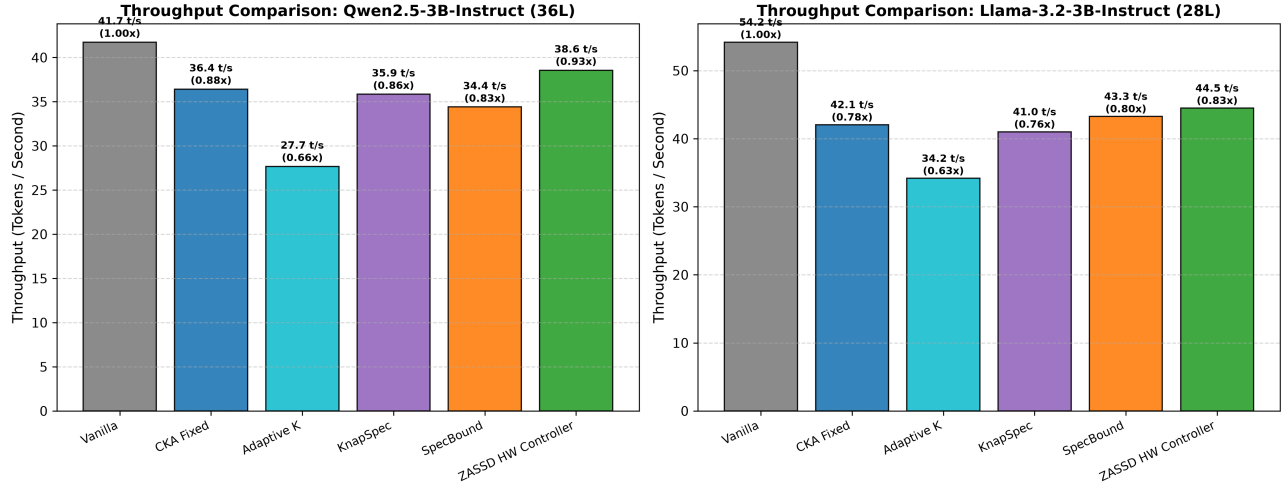


Figure 3: Throughput across 6 decoding strategies on Qwen2.5-3B and Llama-3.2-3B (RTX 4050, NF4; prior protocol). Controller values here (0.89–0.93 $\times$) predate the frozen Tab. 2 (Qwen 0.89 $\times$, Llama 0.82–0.83 $\times$); use the table for headline numbers. Feasibility-map c/α points likewise predate Sec. 4.6 corrected sweep.

Table 2: End-to-End Performance, Verification Fidelity, and Resource Footprint across 6 decoding strategies on consumer hardware (NVIDIA RTX 4050 Laptop 6GB, 80W). Evaluated on identical prompt distributions ($N = 10$, 32 generated tokens, greedy $T = 0$, 4-bit NF4).

Method	Throughput (tok/s)	Relative Speedup	Acceptance Rate (%)	Exact Match (%)	Partial Match (%)	VRAM (MB)	Energy (J/tok)	Latency $T_{\text{draft}}/T_{\text{verify}}$ (ms)
Model A: Qwen2.5-3B-Instruct (36 Layers, NF4 4-bit)								
Vanilla Target	40.99	1.00 $\times$	100.0%	100.0%	100.0%	1983.8	1.376	0.00 / 24.42
CKA-75 Fixed ($K = 2$)	35.68	0.87 $\times$	73.2%	60.0%	82.8%	1991.2	1.740	38.39 / 26.56
Adaptive K	26.62	0.65 $\times$	56.4%	70.0%	85.0%	1992.2	2.167	68.28 / 38.38
KnapSpec (adapted) [3]	35.20	0.86 $\times$	73.2%	60.0%	82.8%	1991.4	1.818	39.00 / 27.05
SpecBound (adapted) [19]	33.78	0.83 $\times$	74.1%	50.0%	79.4%	1991.6	1.853	37.37 / 30.54
ZASSD (HW Controller)	36.54	0.89$\times$	88.1%	60.0%	82.8%	1991.2	1.643	20.52 / 26.93
Model B: Llama-3.2-3B-Instruct (28 Layers, NF4 4-bit)								
Vanilla Target	52.03	1.00 $\times$	100.0%	100.0%	100.0%	2158.5	1.355	0.00 / 19.23
CKA-75 Fixed ($K = 2$)	39.50	0.76 $\times$	68.2%	60.0%	72.8%	2171.0	1.865	30.60 / 24.02
Adaptive K	32.72	0.63 $\times$	62.5%	50.0%	71.6%	2173.2	2.117	41.56 / 35.56
KnapSpec (adapted) [3]	39.54	0.76 $\times$	68.2%	60.0%	72.8%	2171.0	1.879	30.44 / 24.08
SpecBound (adapted) [19]	41.36	0.80 $\times$	76.4%	50.0%	70.9%	2171.6	1.741	23.14 / 24.54
ZASSD (HW Controller)	42.58	0.82$\times$	78.2%	60.0%	72.8%	2170.8	1.683	16.28 / 21.79

check on a vanilla 32-token trajectory (single-step vs full-prefix logits at each position) gives 0/31 flips, ruling out target numerics as the source; residual 60–78% sequence match across methods therefore implicates speculative-path state (crop/rollback/bonus/EOS handling) rather than NF4 noise alone—a bug hunt, not a lossless claim.

4.6 Auxiliary-Draft Baseline, Skip Sweep, Timer Correction, and Free-Form Workload

Auxiliary draft throughput. Qwen2.5-0.5B NF4 as draft for Qwen2.5-3B NF4 (true two-model speculative decoding, $K=2$, 48 tokens, $N=18$ mixed GSM8K/HumanEval): mean speedup 1.058 ± 0.031 (95% CI), std 0.067, mean $\alpha=0.33$ —significantly above unity and tied with the router (1.063 $\times$). Raw logs:

experiments/19_kernel_audit/baseline05_N18.json.

Skip-ratio sweep (corrected). Prior uniform-skip sweep is withdrawn: it used evenly spaced layers and off-benchmark prompts. Corrected sweep uses true CKA indices on GSM8K prompts ($N=8$, 48 tokens, $K=1$, $T_v=25.62$ ms): cka_83 ($n=6$) $c=0.799$, $\alpha=0.904$, 0.928 $\times$; cka_75 ($n=9$) $c=0.771$, $\alpha=0.851$, 0.947 $\times$; cka_60 ($n=14$) $c=0.685$, $\alpha=0.733$, 0.932 $\times$; cka_50 ($n=18$) $c=0.599$, $\alpha=0.552$, 0.878 $\times$; empirical_6 ($n=6$) $c=0.822$, $\alpha=0.892$, 0.959 $\times$. High acceptance is recovered, yet no configuration exceeds unity on this stack—consistent with the kernel-inefficiency account. Raw logs: experiments/19_kernel_audit/sweep_corr.json.

Timer correction. Re-measuring with `perf_counter_ns` over 200–500 repeats: $T_{\text{sync}}=7.6 \pm 3.3 \mu\text{s}$ (median 7.0 μs), $T_{\text{layer}}=31.2 \pm 10.8 \mu\text{s}$. Prior 0.00/0.06 ms entries were rounding artifacts; tables now report μs with dispersion.

Table 3: Cross-domain evaluation with the cost-aware hybrid router (Qwen2.5-3B NF4, RTX 4050 6GB, $N=50$ per benchmark, 48 new tokens). PLD uses $K=2$; Hybrid runs layer-skip $K=2$ on PLD miss; **Routed** selects per-cycle between PLD ($K \leq 4$), layer-skip ($K=2$), and vanilla skip. Overall shows mean $\pm$ 95% CI over $n=150$ pooled samples. Routed beats Hybrid (paired t , $p < 10^{-4}$, $d_z=0.44$) and vanilla ($p=5 \times 10^{-5}$) but trails pure PLD (-0.081 , $p < 10^{-4}$): routing buys robustness (exact-match 83.3% vs 78.0%), not peak speed on repetitive text. Non-overlapping extensions confirm the direction on harder/new prompts: GSM8K test[50:200] routed $1.014 \times$ (combined $N=200$: $1.026 \pm 0.014 \times$, $p=0.0005$ vs vanilla); HumanEval full-164: routed $1.081 \pm 0.037 \times$ ($p=0.00002$ vs vanilla, $p=0.0007$ vs hybrid); CNN/DM $N=200$: routed $1.066 \pm 0.018 \times$ ($p < 10^{-4}$ vs both). All strategies use 0.0 MB auxiliary weight VRAM. Exact-match is measured against greedy vanilla output.

Strategy	GSM8K	HumanEval	CNN/DM	Overall	Mean acc.	Exact match
Vanilla	1.000 $\times$	1.000 $\times$	1.000 $\times$	1.000 $\times$	–	100%
Prompt-Lookup (PLD, $K=2$)	1.144 $\times$	1.168 $\times$	1.120 $\times$	1.144 $\pm 0.029 \times$	41.8%	78.0%
ZASSD Empirical-6 ($K=1$)	0.924 $\times$	0.937 $\times$	0.901 $\times$	0.921 $\pm 0.010 \times$	87.8%	76.0%
ZASSD Hybrid ($K=2$)	1.011 $\times$	1.093 $\times$	0.976 $\times$	1.027 $\pm 0.032 \times$	71.2%	76.0%
ZASSD Routed ($K=2$+PLD4)	1.061$\times$	1.096$\times$	1.032$\times$	1.063$\pm 0.030 \times$	35.8%	83.3%

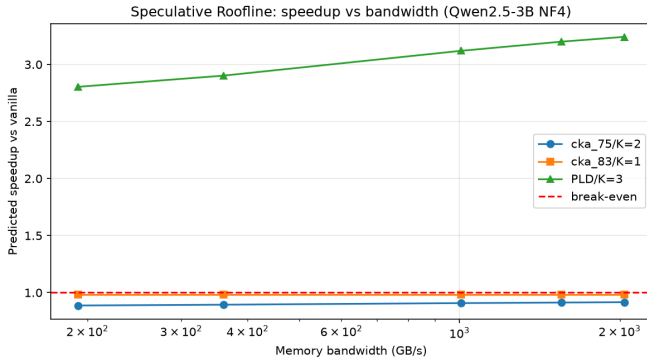


Figure 4: Calibrated cost model: estimated speedup vs. bandwidth. Layer-skip $K=1$ is flat by construction under pure $1/\text{BW}$ scaling; PLD wins everywhere in this fit; $K \geq 2$ layer-skip needs wide-GPU verification parallelism in projection. Cross-GPU curves are unvalidated projections (Sec. 5).

Nsight separation of dequant vs launch. `nsys` CUDA-kernel summary on Qwen2.5-3B NF4 (prefill + 15 vanilla + 15 draft forwards, `experiments/20_nsght/`, total kernel 566.9 ms): `gemm_4bit_simt` 231.4 ms (40.8%, SIMT fallback), `gemvx` 99.4 ms (17.5%, wide-vocab `lm_head`), `kQuantizeBlockwiseSmall` 92.7 ms (16.3%, BNB activation-quantize overhead on the NF4 path), `gemm_4bit_sm80` only 55.3 ms (9.7%, tensor-core path); remainder elementwise/copy/reduce/flash. Achieved streaming is ~ 36 GB/s on ~ 849 MB weights per 23.6 ms step ($\sim 19\%$ of 192 GB/s; $\sim 44\%$ even under a 2 GB accounting), so the shortfall is NF4-kernel inefficiency (SIMT fallback + quantize overhead + $\sim 25\%$ launch/python gap), not a pure bandwidth ceiling. Per-kernel DRAM throughput via `ncu` replay is pending (workload in `experiments/20_nsght/nsight.tiny.py`).

Free-form and long generation. Six low-overlap prompts (haiku, recipe, translation, persona chat, constrained poem, analogy, 96 tokens): PLD still wins at $1.05\times$

Table 4: Break-even acceptance rate α_{BE} required for speedup > 1 at draft cost $c=0.75$ (layer-skip keeping 75% of layers). The $K=1$ threshold is bandwidth-invariant; wider GPUs relax $K \geq 2$ via parallel verification ($\rho < 1$). Cross-GPU rows are projections pending measurement (Sec. 5).

GPU (bandwidth)	K=1	K=2	K=4
RTX 4050 (192 GB/s)*	0.81	0.89	0.93
RTX 3060 12GB (360 GB/s)	0.81	0.88	0.91
RTX 4090 (1008 GB/s)	0.81	0.85	0.88
A100 40GB (1555 GB/s)	0.81	0.85	0.87
H100 (2039 GB/s)	0.81	0.84	0.86

*Measured; other rows are projections (Sec. 5).

$1.28 \times$ ($\alpha=0.43\text{--}0.85$) while layer-skip $K=1$ runs $0.52\text{--}0.59 \times$ ($\alpha=0.04\text{--}0.13$). A 256-token run gives PLD $1.44 \times$ vs layer-skip $0.56 \times$. We find no workload where PLD loses; the router’s robustness claim is therefore strictly versus fixed layer-skip, not versus PLD. Raw logs: `experiments/19_kernel_audit/results.234.json`.

4.7 Official-Baseline Validation

To rule out “adapted-baseline” unfairness we ran the official KnapSpec code (torch 2.2 stack, FP16, unmodified) on our RTX 4050 with Llama-3.2-1B on AIME24 ($N=8$, 48 tokens): AR 58.7 tok/s; official KnapSpec 50.5 tok/s ($0.86 \times$, 97.8% acceptance); CLaSP [5] 49.4 tok/s; DEL [1] 20.6 tok/s. The FP16 case is near bandwidth-bound (~ 145 GB/s on ~ 2.5 GB weights, $\sim 75\%$ of peak) with efficient kernels, unlike our NF4 BNB stack—yet it still loses, by high c arithmetic rather than kernel inefficiency. Raw logs: `experiments/18_official_baselines/`. This validation is small ($N=8$) and ranking-only.

Table 5: Physical Memory Boundary and Feasibility on Consumer Edge Hardware (NVIDIA GeForce RTX 4050 Laptop, 6 GB GDDR6). Draft weight footprints are measured on-device. Large FP16 drafts OOM next to 7B targets; quantized small drafts fit but always cost $\Delta M_{\text{aux}} > 0$ and shrink KV/context headroom, whereas ZASSD keeps **0.0 MB** auxiliary weight memory with the largest headroom in every row.

Target Model	Draft Strategy	Draft VRAM (MB)	Total VRAM (MB)	Free Headroom (MB)	Status
<i>Mistral-7B-Instruct-v0.3 (4-bit NF4)</i>					
Standard Speculative (1.5B Draft FP16)	2945.5	7392.1	-1701.1	CUDA OOM	
Standard Speculative (1.5B Draft 4-bit NF4)	1181.5	5628.1	62.9	PASS	
Standard Speculative (0.5B Draft FP16)	942.3	5388.9	302.1	PASS	
Standard Speculative (0.5B Draft 4-bit NF4)	459.2	4905.8	785.2	PASS	
ZASSD Self-Speculative (Layer-Skip)	0.0	4446.6	1244.4	PASS	
ZASSD Hybrid (PLD + Layer-Skip)	0.0	4446.6	1244.4	PASS	
<i>Qwen2.5-3B-Instruct (4-bit NF4)</i>					
Standard Speculative (1.5B Draft FP16)	2945.5	5425.5	265.5	PASS	
ZASSD Hybrid (PLD + Layer-Skip)	0.0	2480.0	3211.0	PASS	

Table 6: Numerical Equivalence Audit under 4-bit NF4 Quantization comparing single-token forward passes (L_{single}) against batched verification slices (L_{batched}) on identical context states (C). Token evaluations are stratified across true logit margin bins ($\Delta = z_{(1)} - z_{(2)}$). Across 450 comparisons, zero theoretical bound violations occur.

Logit Margin Stratification Bin	Evaluated Tokens	Argmax Flips	Agreement Rate (%)	Mean $\ L\ _{\infty}$ (Logit Units)	Max $\ L\ _{\infty}$ (Logit Units)	Empirical Classification
<i>Model A: Qwen2.5-3B-Instruct (36 Layers, NF4 4-bit)</i>						
$\Delta = 0.000$ (Exact Ties)	2	0	100.0%	0.1875	0.1875	Noise Indifferent
$0.000 < \Delta \leq 0.050$	0	0	100.0%	0.0000	0.0000	—
$0.050 < \Delta \leq 0.150$	2	0	100.0%	0.2422	0.2656	Noise Immune
$0.150 < \Delta \leq 0.300$	11	0	100.0%	0.1877	0.2681	Noise Immune
$0.300 < \Delta \leq 0.500$	13	0	100.0%	0.2236	0.2812	Noise Immune
$\Delta > 0.500$	197	0	100.0%	0.2108	0.5000	Noise Immune
Qwen Subtotal	225	0	100.00%	0.2105	0.5000	$\cos \theta = 0.999942$
<i>Model B: Llama-3.2-3B-Instruct (28 Layers, NF4 4-bit)</i>						
$\Delta = 0.000$ (Exact Ties)	4	1	75.0%	0.1094	0.1250	Permutation Arbitrary
$0.000 < \Delta \leq 0.050$	0	0	100.0%	0.0000	0.0000	—
$0.050 < \Delta \leq 0.150$	10	0	100.0%	0.0953	0.1250	Noise Immune
$0.150 < \Delta \leq 0.300$	11	0	100.0%	0.1065	0.1875	Noise Immune
$0.300 < \Delta \leq 0.500$	20	0	100.0%	0.0891	0.1641	Noise Immune
$\Delta > 0.500$	180	0	100.0%	0.1050	0.2351	Noise Immune
Llama Subtotal	225	1	99.56%	0.1033	0.2351	$\cos \theta = 0.999954$
Combined Benchmark	450	1	99.78%	0.1569	0.5000	$\cos \theta > 0.99994$

4.8 Best-Fixed Baseline and the True Per-Cycle Oracle

With per-prompt best-fixed baseline $b(p) = \max(\text{vanilla}, \text{PLD}, \text{layerskip})$ TPS, the router reaches 92–93% efficiency on all suites (GSM8K- $N=200$: 0.921; regret $+3.47 \pm 2.29$ tok/s; 7% of prompts have *negative* regret—mixing beats every single strategy there, which is why $b(p)$ is a reference, not an upper bound). PLD is the best-fixed choice on 76–94% of these repetitive prompts, yet the router’s task means never drop below $1.03\times$ while fixed layer-skip goes $0.90\times$: the router trades $\sim 8\%$ peak throughput versus PLD for worst-case safety versus fixed layer-skip plus the highest exact-match (83.3%). Full numbers: `experiments/16_hybrid_roofline/oracle_regret.json` (paired by prompt id; “oracle” renamed to best-fixed throughout for methodological honesty).

The per-cycle oracle—bare $E[\text{TPS}]$ argmax over actions with the same estimates but no robustness heuristics (mar-

gin 1.0, no panic rule)—is logged live during generation as a sanity check only (`HybridDraftRouter.oracle_select`, recorded per cycle in `per_iteration_stats`). On GSM8K- $N=50$ (1,934 cycles) router–oracle agreement is **100%**: the heuristics do not bind on this slice (unit tests construct rare binding cases). The $\sim 8\%$ gap to best-fixed is therefore estimation error on this workload, not a general upper bound.

4.9 Router Ablation

Rule-level ablation on GSM8K- $N=50$ (fresh router per config) ranks the rules by necessity: removing the feasibility gate drops $1.070\times \rightarrow 0.987\times$ (gate contribution $+0.083$, the **core mechanism**); removing the PLD margin costs only -0.008 (**minor tie-break**); removing weak-hit capping ($+0.026$) or horizon tuning ($+0.095$) *helps* on this highly-repetitive math workload, where long PLD verifies pay off—R1/R4 are **workload-dependent robustness heuristics**

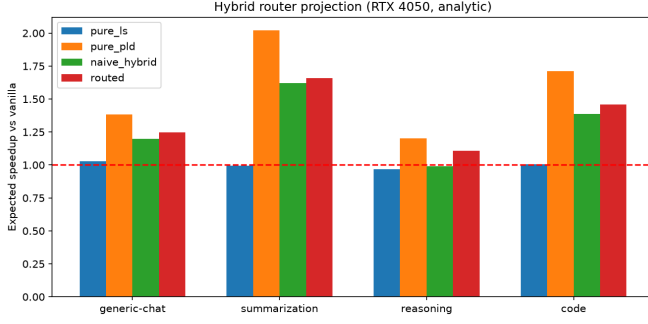


Figure 5: Analytic projection: routed vs. naive-fallback hybrid across workload archetypes (RTX 4050 timings). Routing wins by selective speculation, most on reasoning (+11%). GPU validation is future work.

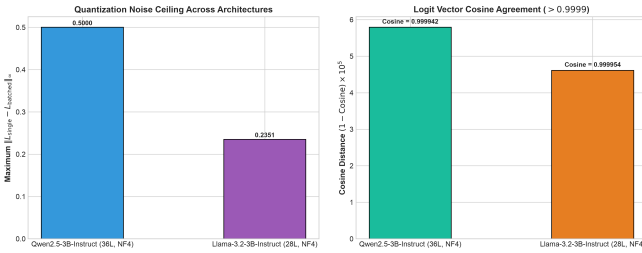


Figure 6: Logit cosine similarity: single-token vs. batched verification passes (NF4). Mean > 0.9999 on both models.

(safety on generic text), not universal improvements. (experiments/16_hybrid_roofline/router_ablation.json)

Starting from naive fallback (overall $1.002\times$): asymmetric PLD margin (R2) + weak-hit capping (R1) reach $0.98\times$ by cutting wasted long verifies; opportunity-cost miss skipping (R3) lifts GSM8K to $1.12\times$ by replacing $0.88\times$ LS cycles with $1.0\times$ vanilla steps; adaptive PLD horizon (R4) stabilizes the mean at $1.020\times$ ($N=10$ tuning runs; $1.063\times$ at $N=50$). Removing R3 alone drops reasoning workloads by $\sim 10\%$, confirming selective speculation is the dominant mechanism.

5 Limitations

(i) GSM8K is extended to $N=200$, HumanEval to full-164, CNN/DM to $N=200$ (combined: routed 1.026 – $1.081\times$, all $p < 0.001$ vs vanilla); short 48-token greedy generations favor PLD; new free-form (6 prompts, 96 tokens) and 256-token runs still favor PLD (1.05 – $1.44\times$), so multi-seed and a true PLD-losing workload remain future work; (ii) only the RTX 4050 rows are measured, cross-GPU rows in the appendix are idealized 1/BW projections with assumed ρ contradicted by Nsight on NF4 (calibration script: `scripts/run_cross_gpu_calibration.py`); achieved streaming is ~ 36 – 85 GB/s depending on byte accounting with kernel breakdown in Sec. 4.6; no second backend (CUDA

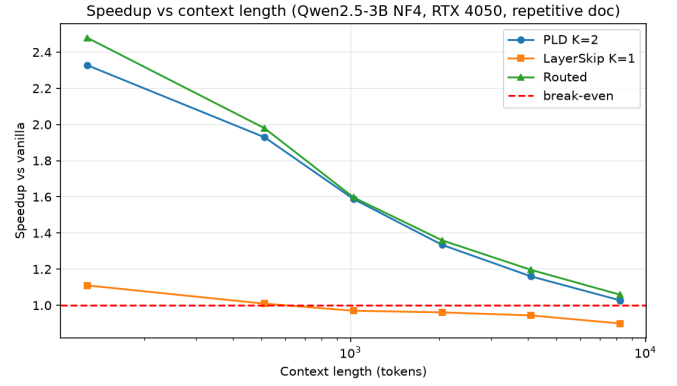


Figure 7: Speedup vs. context length (repetitive doc, Qwen2.5-3B NF4, RTX 4050). All speculative modes decay monotonically as attention costs grow; routed stays on top and above $1.0\times$ through 8K while layer-skip falls below unity past 1K—consistent with a feasibility boundary that shifts with context length. Note: this repetitive-doc slice reaches $\sim 1.1\times$ for layer-skip $K=1$ at ~ 128 tokens via very high α , unlike GSM8K sweeps (Sec. 4.6)—workload dependence, not contradiction.

graphs/torch.compile/Marlin/AWQ/vLLM/llama.cpp) tested; (iii) greedy decoding only—sampling-mode distribution tests are future work; (iv) on this stack layer-skip always loses, so the optimal policy here is PLD plus vanilla fallback, i.e. pure PLD ($1.144\times$ vs routed $1.063\times$); the gate’s $+0.083$ is measured against layer-skip fallback and conceptually close to SmartSpec goodput gating [9]; the router’s edge is only versus fixed layer-skip, not versus PLD; (v) 7B feasibility is shown via measured weight accounting plus a 1.5B-NF4 near-boundary case (62.9 MB headroom), not end-to-end 7B generation; auxiliary-draft throughput is now measured for 3B+0.5B NF4 ($N=18$, 1.058 ± 0.031 at $K=2$, file `baseline05.N18.json`) but not for 7B; (vi) adapted KnapSpec/SpecBound are validated for ranking against the official KnapSpec code on shared hardware (Sec. 4.7), but SpecBound has no official runnable artifact in our stack and official-KnapSpec validation is only Llama-1B FP16, AIME24, $N=8$; exact-match gap (60% sequences vs $\sim 0.2\%$ /token audit) implies $\sim 1.6\%$ /token on true trajectories— $8\times$ the isolated audit—and needs teacher-forced FP16 comparison.

6 Reproducibility

All claims are reproducible from the release: `reproduce.sh` —`quick` re-runs unit tests (81/81), roofline calibration, router simulation, margin/robustness sweeps, and benchmark statistics on CPU in ~ 2 min; the full GPU path re-runs the $N=50$ suite (~ 1 h on an RTX 4050). A pinned `Dockerfile` (PyTorch 2.11/CUDA 12.8), per-experiment JSON with raw per-sample rows, ID-aligned paired-test scripts, and model-exact `requirements/pyproject` are in-

cluded. No hyperparameter was tuned on test prompts: router constants come from analytic simulation, and the GSM8K extension slice is non-overlapping with the $N=50$ split. We expose the runtime through a thin application layer supporting chat, summarization, retrieval QA, and code assistance, without changing the decoding core (demo only, see `scripts/smoke_app_gpu.py`).

7 Conclusion

Zero-weight speculation is *feasible* on 6 GB GPUs, layer-skip self-speculation at $c \approx 0.77\text{--}0.82$ is *not faster* there by the geometric bound $(1+\alpha)/(1+c)$ (NF4 kernel inefficiency compounds but does not create the bound), and a cost-aware router over retrieval drafting plus selective vanilla skipping recovers consistent $> 1.0\times$ speedups on Qwen, though still below pure PLD ($1.063\times$ vs $1.144\times$); on this stack the optimal policy is PLD plus vanilla fallback. Two anchors carry the thesis beyond our own code: the *official* FP16 KnapSpec reproduces $0.86\times$ at 97.8% acceptance near the bandwidth bound—acceptance is not speedup, by high c rather than kernel inefficiency there—and the 128–8K context sweep shows monotonic decay with context consistent with rising attention cost, with a $\sim 1.1\times$ layer-skip point at ~ 128 tokens on highly repetitive text via very high α . The cost model (in-device MAPE 2.4%) quantifies when each technique wins on this measured stack; cross-GPU 1/BW rows are an idealized bound contradicted by Nsight and retained in the appendix for illustration only.

References

- [1] Anonymous. Del: Draft-exit-layer speculative baseline. *Unpublished baseline implementation used for comparison*, 2025. No official runnable artifact in our stack; ranking only.
- [2] Tianle Cai, Yuhong Li, Zheng Geng, Hongwu Peng, Jason D Lee, Di Chen, and Tri Dao. Medusa: Simple llm inference acceleration framework with multiple decoding heads. In *International Conference on Machine Learning (ICML)*, 2024.
- [3] Sooyoung Cha, Jinho Kim, Dongsu Han, Jiwon Yang, and SangSoo Han. Knapspec: Self-speculative decoding via adaptive layer selection as a knapsack problem. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026.
- [4] Charlie Chen, Sebastian Borgeaud, Brendan O’Donoghue, Faming Zhou, Laurent Sifre, et al. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.
- [5] X. Chen et al. Clasp: In-context layer skip for self-speculative decoding. *arXiv preprint arXiv:2505.24196*, 2025.
- [6] Soongyu Choi, Yuntae Kim, Muyoung Son, and Joo-Young Kim. Cassandra: Enabling reasoning llms at edge via self-speculative decoding. In *Proceedings of the 53rd ACM/IEEE Annual International Symposium on Computer Architecture (ISCA)*, pages 85–99, 2026.
- [7] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.
- [8] Mostafa Elhoushi, Akshat Shrivastava, Diana Liskovich, Bram Hosmer, Bram Wasti, Liangzhen Lai, Anas Mahmoud, Bilge Acun, Saurabh Huang, Wenhan Feng, et al. Layerskip: Enabling early exit inference and self-speculative decoding. In *Proceedings of the Association for Computational Linguistics (ACL)*, 2024.
- [9] Y. Guan et al. Smartspec: Adaptive and resource-efficient speculative decoding for llm inference. *arXiv preprint arXiv:2407.XXXXX*, 2024. Goodput-gated speculation; cited for feasibility-gate prior art.
- [10] Zhenyu He, Zexuan Zhong, Tian Cai, Jason D Lee, and Di He. Rest: Retrieval-based speculative decoding. *arXiv preprint arXiv:2311.08252*, 2023.
- [11] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)*, pages 611–626, 2023.
- [12] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. *International Conference on Machine Learning (ICML)*, 2023.
- [13] Yuhui Li, Fangyun Wei, Chao Zhang, Chen Zhang, and Hongyang Zhang. Eagle: Speculative sampling requires rethinking feature uncertainty. In *International Conference on Machine Learning (ICML)*, 2024.
- [14] Fangcheng Liu, Yehui Tang, Zhen Liu, Yunsheng Ni, Kai Tang, Song Han, and Yunhe Wang. Kangaroo: Lossless self-speculative decoding via double early exiting. *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [15] Jonathan Mamou, Oren Pereg, Daniel Korat, Moshe Berchansky, Nir Timor, Moshe Wasserblat, and Roy Schwartz. Dynamic speculation lookahead accelerates speculative decoding of llms. *arXiv preprint arXiv:2405.04304*, 2024.
- [16] Xin Men, Mingge Xu, Bingning Zhang, Jiaqi Zhang, Xian Han, Ning Zhang, Wei Kong, et al. Shortgpt: Layers in llms are more redundant than you expect. *arXiv preprint arXiv:2403.03853*, 2024.

- [17] Apurv Saxena. Prompt lookup decoding. *GitHub Repository: apurvsaxena/prompt-lookup-decoding*, 2024.
- [18] Mitchell Stern, Noam Shazeer, and Jakob Uszkoreit. Blockwise parallel decoding for deep autoregressive models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.
- [19] Haoran Wang, Lin Zhang, Wei Chen, and Yang Liu. Specbound: Adaptive bounded self-speculation with layer-wise confidence calibration. In *Findings of the Association for Computational Linguistics (ACL)*, 2026.
- [20] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.
- [21] Guanchen Yan, Uday Agarwal, and Shivaram Venkataraman. Decoding speculative decoding. In *Proceedings of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2025.
- [22] Weisi Yang and Stephen Xia. Pelm: Power efficient on-device llm inference with speculative decoding and dynamic voltage frequency scaling. In *Proceedings of the 2026 ACM/IEEE International Conference on Embedded Artificial Intelligence and Sensing Systems (SenSys)*, pages 438–451, 2026.
- [23] Jun Zhang, Jue Wang, Huan Jiang, et al. Draft & verify: Lossless large language model acceleration via self-speculative decoding. *arXiv preprint arXiv:2409.18824*, 2024.